AMRITA VISHWA VIDYAPEETHAM

AUV Adaptive MPC with Regularized Dynamic-Forgetting Gaussian Processes

A complete MATLAB / Simulink reproduction, correction and extension of the base framework, benchmarked against industry-standard PID control

Presented By

Team-16

23AID305 - Control System

Faculty In Charge - **Dr. Rahul Satheesh S**

Lalith Sagar - CB.AI.U4AID24009

Ruthwik Keetha - CB.AI.U4AID24023

Muthu Raam T - CB.AI.U4AID24058

Prapul Chandra - CB.AI.U4AID24063

School of Artificial Intelligence

Amrita Vishwa Vidyapeetham

Academic Year - 2026 - 27

Introduction & Motivation

The Application of AUVs:

- **Pipeline inspection, seabed mapping and subsea survey.**
- These applications depend on holding an **accurate position** and **heading in a constantly shifting, hard-to-predict environment.**
- Full autonomy demands **accurate, safe navigation** in cluttered underwater environments.

The Two Approaches (Controllers used):

1. The **learning-based MPC** Controller - **base research paper.**
2. The **PID-based** controllers - **Real vehicles actually deploy today.**

This project identifies the flaws in each and proposes a combined body of work that addresses both — entirely in MATLAB and Simulink, without hardware access.

Why MPC, and why MPC alone is not enough?

- **MPC plans ahead over a future horizon** rather than reacting moment to moment, and respects **actuator and state limits** natively as part of its **optimisation.**
- But **MPC is only as good as the model it optimises against,** and **underwater models are poor.**
- **External Disturbance - currents, tether pull, thrust degradation(The unmodelled portion of a vehicle's behaviour)**
- If the controller cannot estimate that disturbance, tracking accuracy degrades exactly when precision matters most.
- A single Gaussian Process with one fixed forgetting factor cannot handle disturbances that change character mid-mission.

Problem Statement

The problems that exist today, and the problems this project solves

1. Underwater vehicles must hold a **precise path** in water whose **disturbances change character mid-mission.**
2. The published **learning-based MPC has structural flaws**
3. **PID control** (Real world vehicles) - **reactive** and has **never been measured against it.**

Problems in the MPC (base paper):

- The **weight-fusion** step is linear in the weights, so the "blend" is really a hard switch between GPs.
- The **predictive variance** is computed every step but never used.
- No **thruster-fault test**
- No **repeated-trial validation**: single-run results only.

Problems in real-world PID

- **Purely reactive:** it acts only after the error appears.
- No **vehicle model** or **disturbance awareness:** it cannot tell a current from a **thrust loss.**
- Limits clamped on afterwards; **axis coupling ignored.**
- Never **benchmarked** against **learning-based MPC.**

Problems solved in this project

- **Regularised QP fusion (ρ = 0.05)** gives a genuine blend: churn 0.049 → 0.045 per step.
- **The variance now tightens** the actuator bounds: uncertainty-aware MPC.
- **Fault test (+31.9% vs +35.0%)**
- **Eight-seed Monte Carlo** (std 0.34 vs 0.66).
- **PID benchmarked** on the same trajectory and actuator clamps.

Literature Review

A. GP-based model learning for MPC

Mehndiratta & Kayacan, ECC 2020	GP learns the disturbance - Using a partitioned static / dynamic dataset.	Limitation: Hyperparameters fixed offline.
Torrente et al., IEEE RA-L 2021	GP captures unmodelled aerodynamic drag on quadrotors for data-driven MPC.	Limitation: Trained offline, no adaptation.

B. GP-MPC with guarantees

Hewing, Kabzan & Zeilinger, IEEE TCST 2020	Cautious MPC that propagates GP uncertainty into the constraints.	Limitation: Computationally heavy.
Maiworm, Limon & Findeisen, IJRNC 2021	Output-feedback MPC with an evolving GP and stability guarantees.	Limitation: Conservative, still offline-tuned.

C. Adaptive / sparse Gaussian processes

Gómez-Verdejo et al., IEEE TNNLS 2024	Adaptive Sparse GP: a forgetting factor λ weights recent data, enabling fast online updates.	Limitation: No rule to choose λ.
Long et al., Ocean Eng. 2021	ESKF-based disturbance observer combined with MPC for tethered ROVs.	Limitation: the estimator has no predictive capability.

Research Gap

01	**Offline hyperparameters**	**Problem** **Measured only once** in the starting **rely on history**	**Proposed work solution** **Sliding buffer** refreshed every **0.2 s; hyperparameters recomputed** from it
02	**No rule for λ**	**Problem** Forgetting factor selection out of the 3	**Proposed work solution** **3 GPs (λ = 1.0, 0.8, 0.6)** blended by a **regularized QP** every step
03	**Underwater multi-timescale**	**Problem** Multi-timescale underwater disturbances remain under-studied	**Proposed work solution** **4-DOF BlueROV2 model** plus a **thruster fault at 45 s**
04	**No field benchmark**	**Problem** Learning-based MPC is not benchmarked against deployed practice	**Proposed work solution** **PID** on the same path and limits: **0.2447 m vs 0.1995–0.2126 m**

Objectives

01	**Fix the blend:**	**Problem** The base paper's blend is **really a switch**	**Solution** Add a **small penalty** so weights are truly shared	**Check** Smoother weights (churn 0.049 → 0.045)
02	**Use the confidence**	**Problem** **GP confidence** is calculated but ignored	**Solution** When the estimate is unsure, **limit the thrust**	**Check** Limits shrink as uncertainty rises, **no violations**
03	**Compare with PID**	**Problem** **Real vehicles use PID**, never compared	**Solution** **Run PID** on the same path and limits	**Check** All **5 methods** in one test
04	**Stress-test**	**Problem** Base paper shows **one run** only	**Solution** Break a thruster at 45 s Repeat with 8 noise seeds	**Check** **Damage vs NoGP**; mean and spread reported

Architecture

- Mission starts - 8 shaped reference path x(reference) is fed to the controller

Start: Reference Path

- Optimise thrust over the next 1.6 s; apply only the first action u_opt
- High uncertainty → tighter thrust limits

Uncertainty-Aware MPC

- Thrust [X, Y, Z, M_z] moves the vehicle while a hidden disturbance Δ pushes it

AUV Plant (Eq. 1–8)

- Runge–Kutta 4 steps the dynamics 0.2 s ahead to give the new state, fed back to Eq. 1–8.

RK4 State Update

- Measured motion - model prediction = the force the model cannot explain.

Residual Disturbance

- Three GPs (λ = 1.0, 0.8, 0.6) learn it from recent data; each gives a mean and a variance.

Forgetting GP Bank

- Weights η from a regularised QP (ρ = 0.05):
- A true blend, not a hard switch.

Regularised QP Blend

- Combine into estimate $\hat{\Delta}$ and confidence $\hat{\Sigma}$; both go back to the MPC.

Fused $\hat{\Delta}$ and $\hat{\Sigma}$

- Condition check: is t ≥ 90 s?
- No → repeat the next 0.2 s step.

Mission over?

No

Yes

- Report tracking RMSE, disturbance RMSE, weight churn and actuator effort.

Output Metrics

End

Methodology

Kinematics (Eq. 1, 2, 3, 7): Turns vehicle-frame speeds into real-world motion.

$$\dot{x} = \cos(\psi)u - \sin(\psi)v$$ ⟷ Gives the vehicle's east-west speed from its body speeds and heading

$$\dot{y} = \sin(\psi)u + \cos(\psi)v$$ ⟷ Gives the north-south speed the same way

$$\dot{z} = w$$ ⟷ Depth changes at the heave speed

$$\dot{\psi} = r$$ ⟷ Heading changes at the yaw rate

What it does	Inputs → Outputs	Status
Rotates body velocities into the earth frame	In: u, v, w, r, heading ψ	Unchanged from base paper
Thrusters act along the vehicle's heading, not the compass	Out: Rate of change of x, y, z, ψ	Feeds RK4 and the MPC prediction

Methodology

Dynamics (Eq. 4, 5, 6, 8): Force → acceleration for surge, sway, heave and yaw.

$$(m - X_{\dot{u}})\dot{u} = X + (mv + Y_{\dot{v}}v)r + (X_u + X_{uc}|u|)u + \Delta_x$$

Surge: Forward acceleration from thrust, Coriolis, drag and the disturbance Δx

$$(m - Y_{\dot{v}})\dot{v} = Y - (mu + X_{\dot{u}}u)r + (Y_v + Y_{vc}|v|)v + \Delta_y$$

Sway: Sideways acceleration from thrust, Coriolis, drag and Δy

$$(m - Z_{\dot{w}})\dot{w} = Z + (Z_w + Z_{wc}|w|)w + mg - V_{\text{sub}}\rho_{\text{water}} + \Delta_z$$

Heave: vertical acceleration from thrust, drag, buoyancy and Δz

$$(I_{zz} - N_{\dot{r}})\dot{r} = M_z - (mv - Y_{\dot{v}}v)u - (X_{\dot{u}}u - mu)v + (N_r + N_{rc}|r|)r + \Delta_{M_z}.$$

Yaw: turning acceleration from the yaw moment, Coriolis, drag and ΔMz

Other equations	Key term	Status
Inertia × Acceleration = Total force	Δ = Unknown disturbance (current, tether, thrust loss)	Unchanged, plus sign fix (Eq. 8 vs 4–5)
Force = Thrust + Coriolis + Drag + Restoring	The GP's job is to estimate Δ	Model: BlueROV2 Heavy

Methodology

State-Space Form (Eq. 9)

All four dynamics equations packed into one function the controller can use.

$$\dot{\mathbf{x}} = \mathtt{f}(\mathbf{x}_k, \mathbf{u}_k, \Delta).$$ ⟷ **Packs Eq. 1–8 into one function: state, thrust and disturbance in, motion out**

$$\mathbf{x} = [x, y, z, u, v, w, \psi, r]^T$$ ⟷ **The 8 states: Position and heading, plus their speeds**

$$\mathbf{u} = [X, Y, Z, M_z]^T.$$ ⟷ **The 4 control inputs: three forces and one yaw moment**

States	Inputs	Unknown
4 positions (x, y, z, ψ) + 4 speeds (u, v, w, r)	3 forces + 1 yaw moment	Δ: the part the GP learns

Methodology

RK4 Prediction (Eq. 12b)

Steps the vehicle model 0.2 s into the future, accurately.

$$\mathbf{x}_{k+1} = \mathrm{f}_d(\mathbf{x}_k, \mathbf{u}_k, \Delta), \quad k \in [0, N_c - 1]$$

Prediction model: RK4 steps the vehicle forward 0.2 s using the disturbance estimate

What it does	Why RK4	Status
Gives the MPC future states to plan with	Four slope checks per step: very accurate	Same formulation as paper
Uses the estimate $\hat{\Delta}$ inside the model	Input held constant over each step	Solver: fmincon (paper: qpOASES)

Methodology

Residual Disturbance (Proposed add-on)

Disturbance = what the vehicle really did - what our model predicted.

$$\dot{\nu}_{\text{nom}} = M^{-1}\left[\tau + C(\nu)\nu + D(\nu)\nu + g\right]$$

$$\dot{\nu}_{\text{meas}} = \frac{\nu_{k+1} - \nu_k}{T_s}$$

$$\Delta\tau_k = M\left(\dot{\nu}_{\text{meas}} - \dot{\nu}_{\text{nom}}\right)$$

What the vehicle actually did vs what the model predicted: the disturbance, with no extra sensor

What it does	Feedback to Eq. 1–8	Status
The leftover force the model cannot explain	New state x_{k+1} goes back into Eq. 1–8 as next step's start	Our own addition
Needs no extra sensor	Same state also gives the GP its input	Becomes the GP target

Methodology

GP Target (Eq. 25)

The GP learns: "in this motion state, how big is the disturbance?"

$$y_i = f(\mathbf{a}_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_{\varepsilon_i}^2)$$

Each observation is the true function plus small noise

$$y = \Delta_\tau.$$

GP target: the disturbance the GP must learn

Target	Input	Our change
y = residual disturbance from the last slide	a = body speeds and yaw rate	Single sliding buffer of recent data
One value per axis	Only current state, no history	Simpler and faster

Methodology

Kernel (Eq. 16)

The kernel measures how similar two situations are.

$$k(\mathbf{a}, \mathbf{a}') = \sigma_f^2 \exp\left(-\frac{1}{2}(\mathbf{a} - \mathbf{a}')^T \mathbf{L}_a^{-2}(\mathbf{a} - \mathbf{a}')\right)$$

Similarity measure: close operating conditions give similar disturbances

Meaning	Parts	Proposed work change
Similar speeds → similar disturbance	σf² = signal size; L = how far "similar" reaches	Hyperparameters set by a direct formula
Gives each input its own length scale	ARD: one scale per input	Paper used slow iterative fitting (Eq. 17)

Methodology

Forgetting Weight (Eq. 22)

Each GP gives old data less weight, at its own speed.

$$\Lambda = \mathrm{diag}\left(\lambda^{n-1} \quad \lambda^{n-2} \quad \cdots \quad \lambda^{0}\right)$$

Similarity measure: close operating conditions give similar disturbances

λ = 1.0	λ = 0.8	λ = 0.6
No forgetting; best for slow, steady disturbances	Medium memory	Forgets fast; best for sudden changes

Our change: Forgetting is applied as a simple per-sample weight (no sparse matrix).

Methodology

GP Posterior (Eq. 18–21)

Each GP outputs a prediction and a confidence.

$$\mu_j(a_*) = k_*^\top \left(K + \sigma_\epsilon^2 \Lambda_j^{-1}\right)^{-1} y$$ ⟷ **Best guess of the disturbance at a new input**

$$\sigma_j^2(a_*) = k_{**} - k_*^\top \left(K + \sigma_\epsilon^2 \Lambda_j^{-1}\right)^{-1} k_*$$ ⟷ **How unsure the GP is about that guess**

$$\mu_j(a_*) = \sum_{i=1}^{N} \beta_{j,i}\, k(a_*, a_i), \qquad \beta_j = \left(K + \sigma_\epsilon^2 \Lambda_j^{-1}\right)^{-1} y$$ ⟷ **Same guess using a small set of inducing points, for speed**

$$\sigma_j^2(a_*) = k_{**} - k_*^\top \left(K + \sigma_\epsilon^2 \Lambda_j^{-1}\right)^{-1} k_*$$ ⟷ **Same confidence using inducing points**

Mean μ	Variance σ²	Our change
Best guess of the disturbance	How unsure the GP is	Dense weighted form replaces sparse Eq. 20–21
One per GP (3 total)	Low = confident	Same idea, simpler to build

Methodology

Weight Blending (Eq. 26a, proposed)

Decide how much to trust each GP

$$\min_{\eta} \sum_{j=1}^{3} \eta_j \, e_j + \rho \, \|\eta\|^2, \qquad \rho = 0.05$$

Chooses how much to trust each GP by their recent errors; our penalty makes it a true blend

Problem with paper	Proposed work fix	Result
Linear cost puts all weight on one GP	Add ρ‖η‖² penalty	Weights are shared between good GPs
"Blend" is really a switch	Constraints unchanged	Churn 0.049 → 0.045
Error = last 20 samples	Solved with quadprog	Weights logged every step to measure churn

Methodology

Fused Mean and Variance (Eq. 28, 29)

Combine the three GPs into one estimate and one confidence

$$\mu_{*\mathrm{DF}} = \sum_{j=1}^{K} \eta_j^{\mathrm{opt}} \mu_{*t}(\lambda_j)$$

Fused estimate: Weighted average of the three GP predictions

$$\Sigma_{*\mathrm{DF}} = \sum_{j=1}^{K} \eta_j^{\mathrm{opt}} \Sigma_{*t}(\lambda_j).$$

Fused confidence: Weighted average of the three variances

$\hat{\Delta}$ goes to	$\hat{\Sigma}$ goes to	Status
MPC prediction model (corrects the dynamics)	Actuator bounds (tightens them when unsure)	Eq. 28 unchanged
Updated every 0.2 s	Paper never used it	Eq. 29 formula same, usage new

Methodology

MPC Solve (Eq. 12a–c) and Closed Loop

Pick the best thrust sequence for the next 1.6 s, then apply the first step.

$$\min_{\mathbf{x}_k,\mathbf{u}_k} \sum_{k=0}^{N_c-1} \left(\|\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}\|_{\mathbf{Q}}^2 + \|\mathbf{u}_k\|_{\mathbf{R}}^2 \right) + \|\mathbf{x}_{N_c} - \mathbf{x}_{N_c}^{\text{ref}}\|_{\mathbf{Q}_T}^2$$

Cost to minimise: tracking error (Q) plus thrust effort (R) over the horizon

$$\mathbf{x}_{k+1} = \mathrm{f}_d(\mathbf{x}_k, \mathbf{u}_k, \Delta), \quad k \in [0, N_c - 1]$$

Prediction model: RK4 steps the vehicle forward 0.2 s using the disturbance estimate

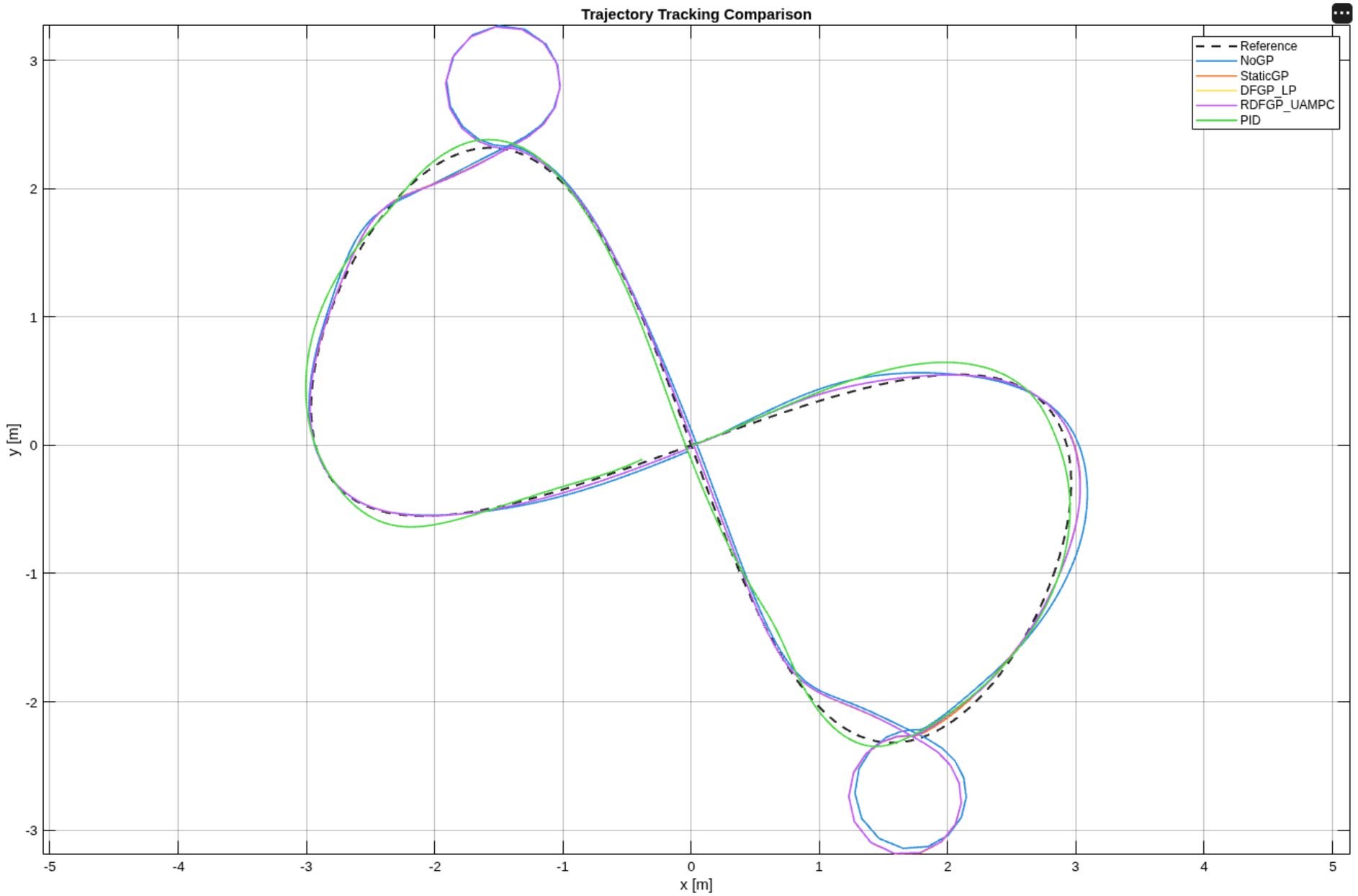
$$\mathbf{u}_{k,\mathrm{l}} \le \mathbf{u}_k \le \mathbf{u}_{k,\mathrm{u}}, \quad k \in [0, N_c - 1].$$

Thrust limits the optimiser must respect

Cost (12a)	Constraints (12b–c)	Closing the loop
Track the path with little thrust	Model and thrust limits respected	u_opt → Eq. 4–8 as the applied force
Horizon: 8 steps × 0.2 s	Limits shrink when $\hat{\Sigma}$ is high	Vehicle moves → repeat every 0.2 s

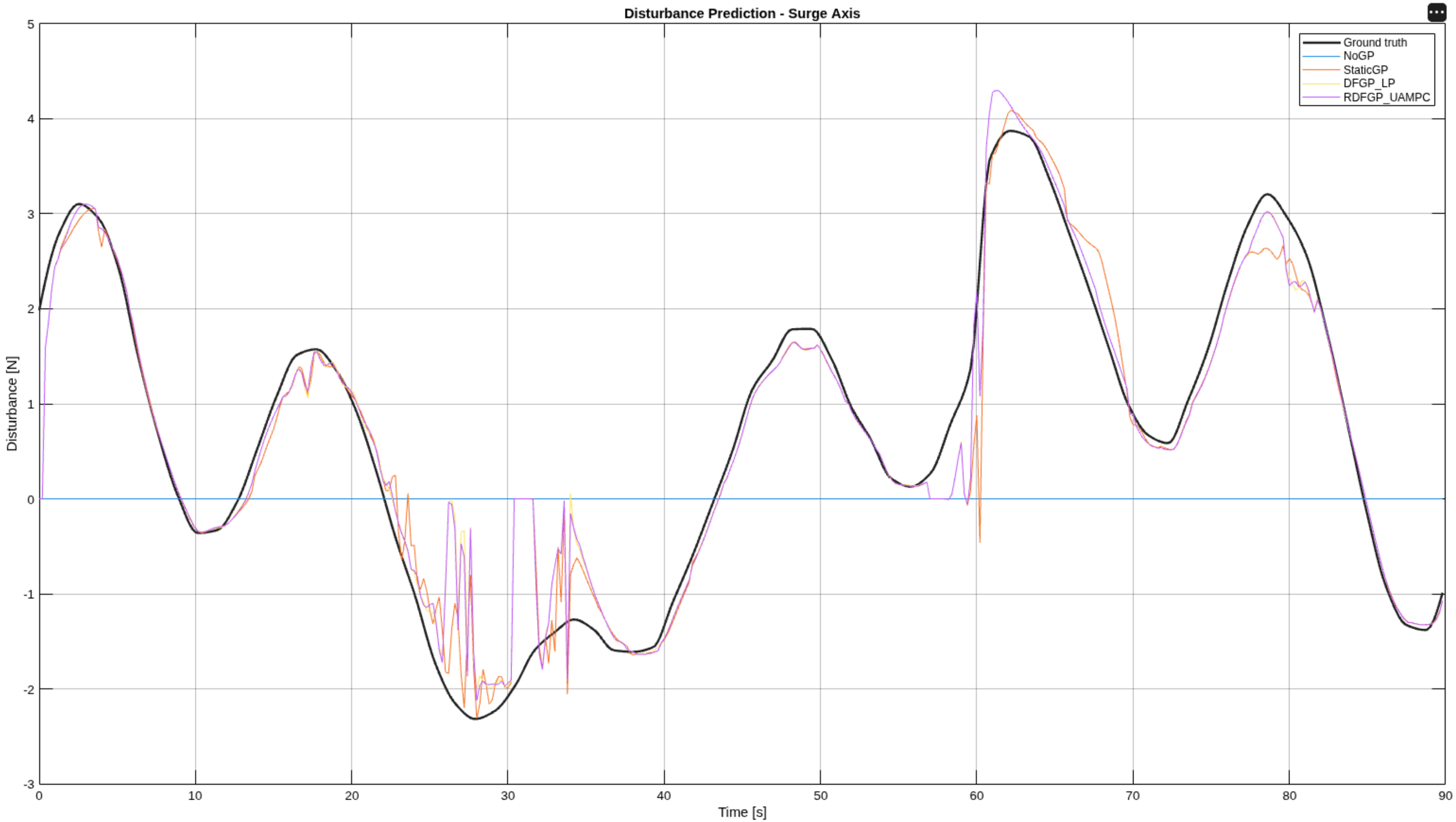
Trajectory Tracking

Shows how closely each controller (PID, NoGP, StaticGP, DFGP_LP, RDFGP_UAMPC) follows the figure-8 reference path.



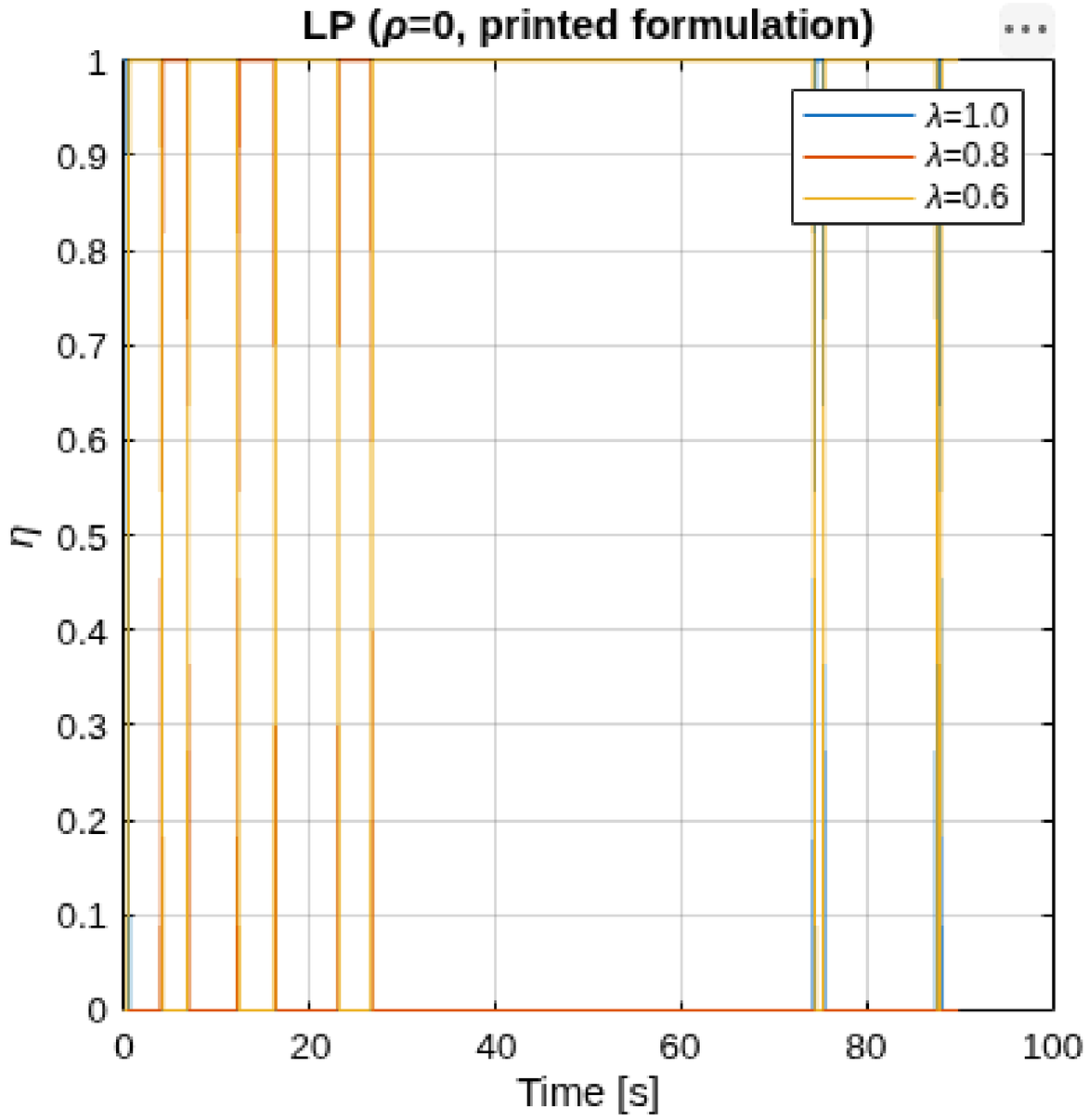
Disturbance Prediction, Surge Axis

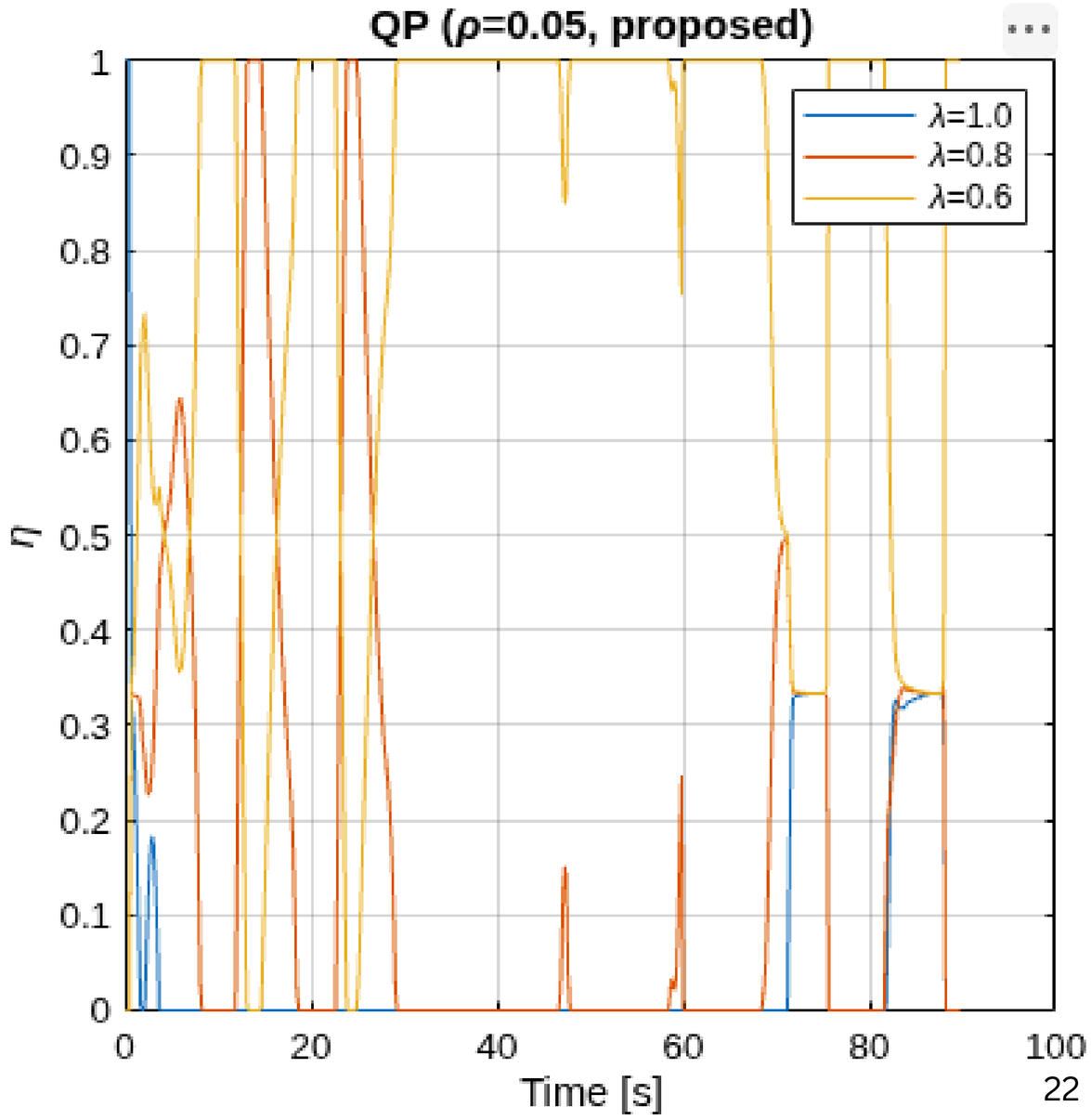
Shows how well the GP estimate follows the true hidden disturbance along the surge axis while the vehicle is running.



LP vs QP Blend Weights

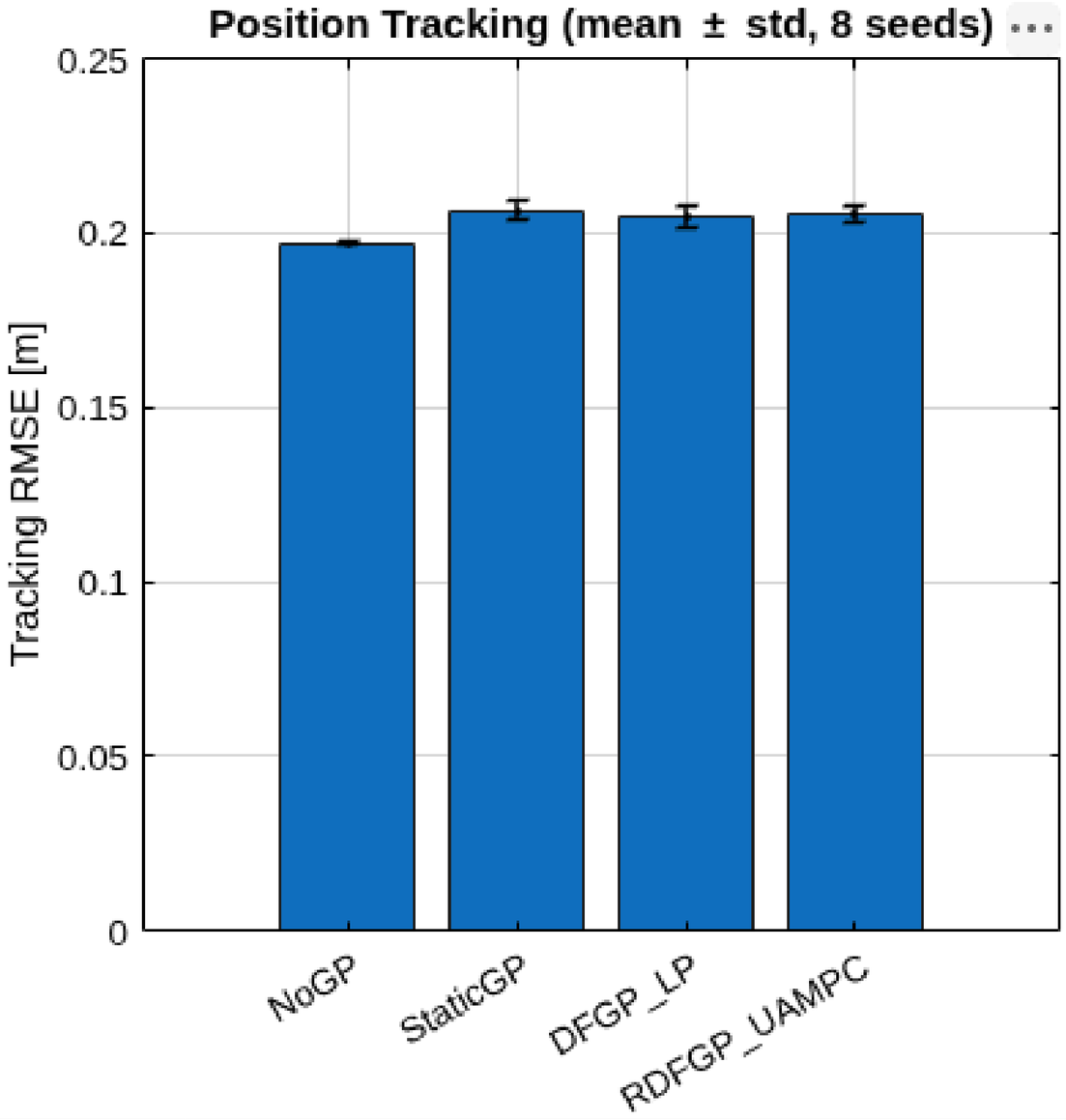
Shows that the base paper's LP switches all the weight between GPs, while our regularised QP shares it smoothly.

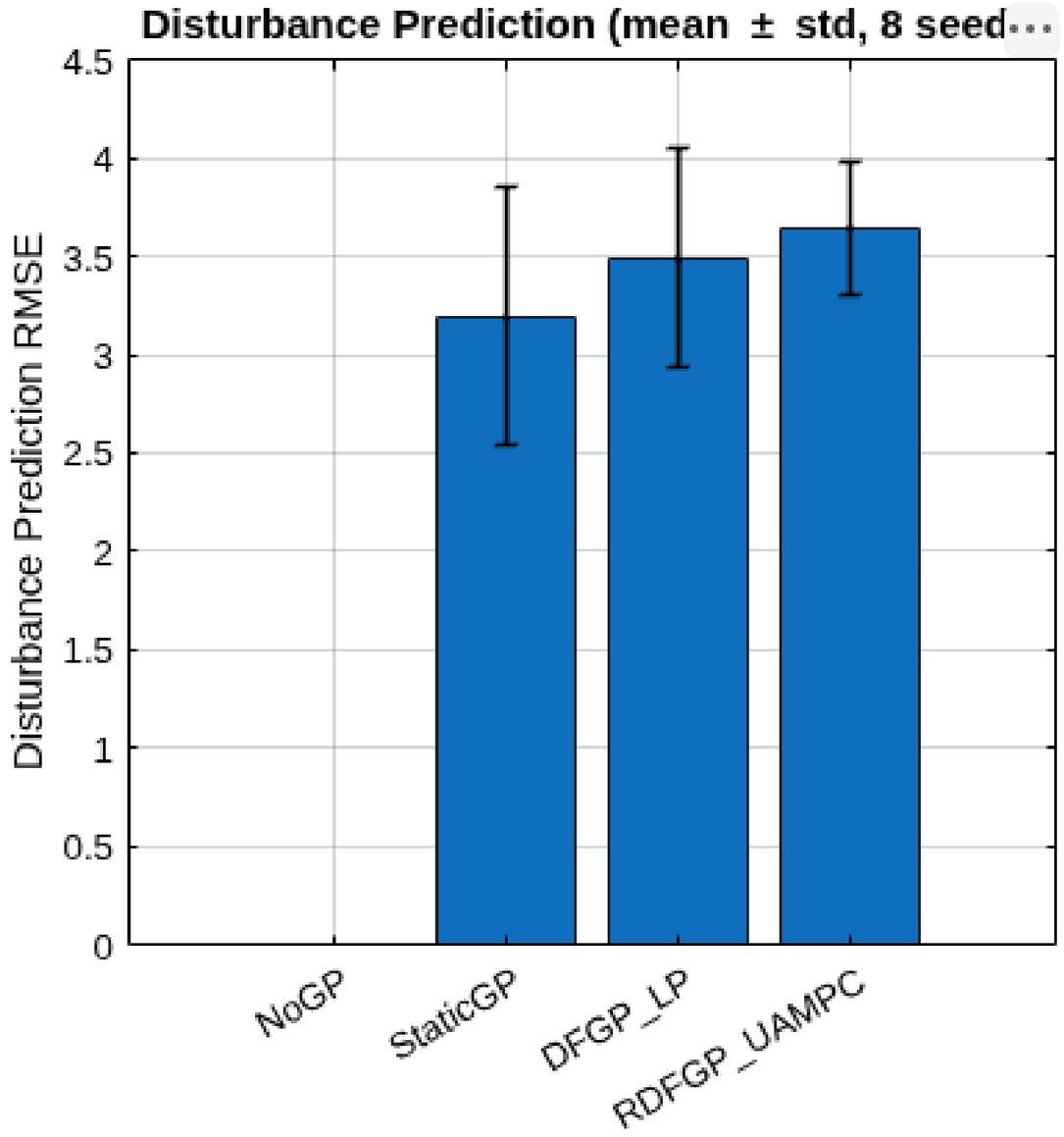




Monte Carlo Robustness

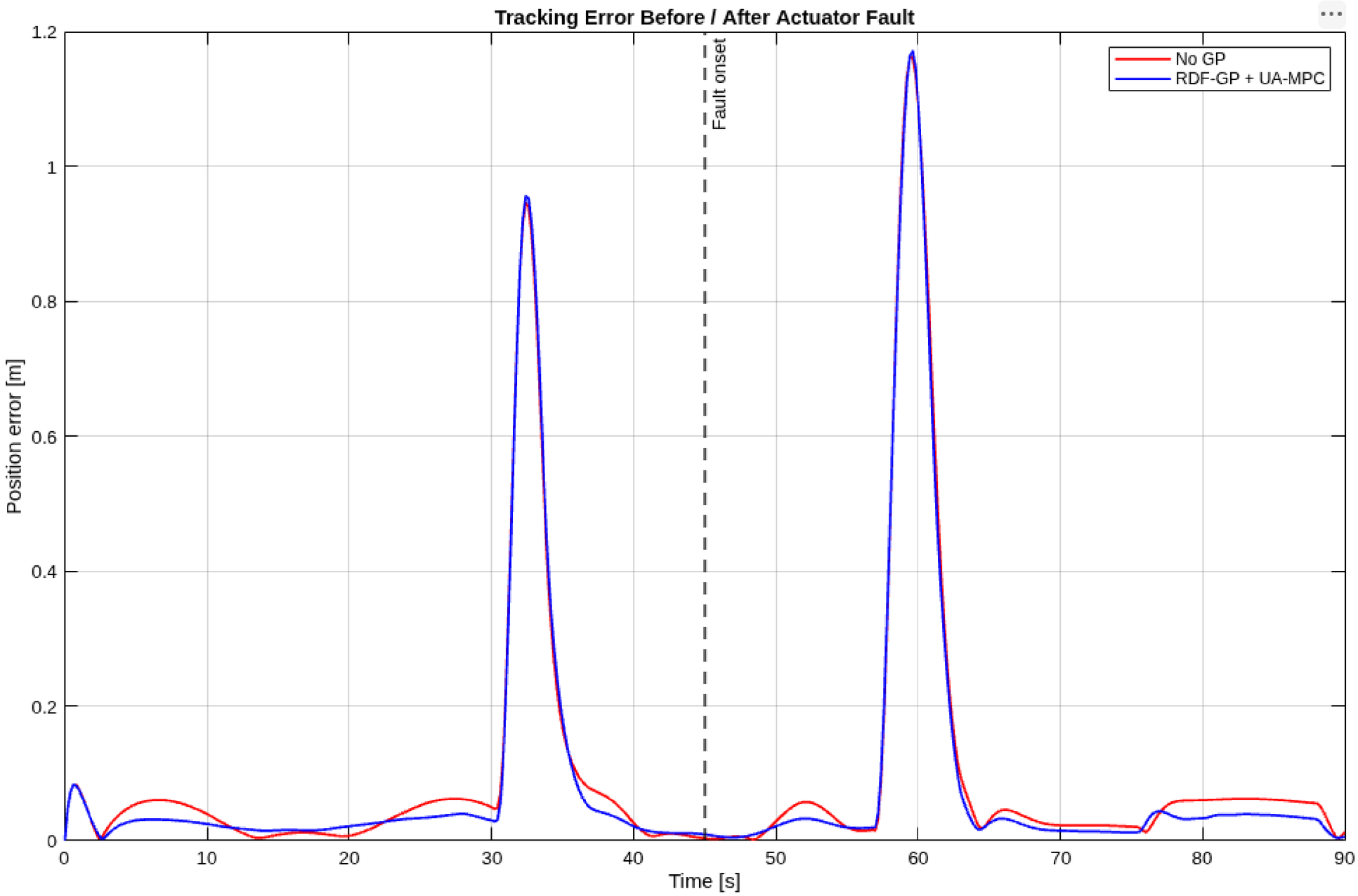
Shows how much each method's error varies across 8 random noise seeds, to test consistency and not just one run.





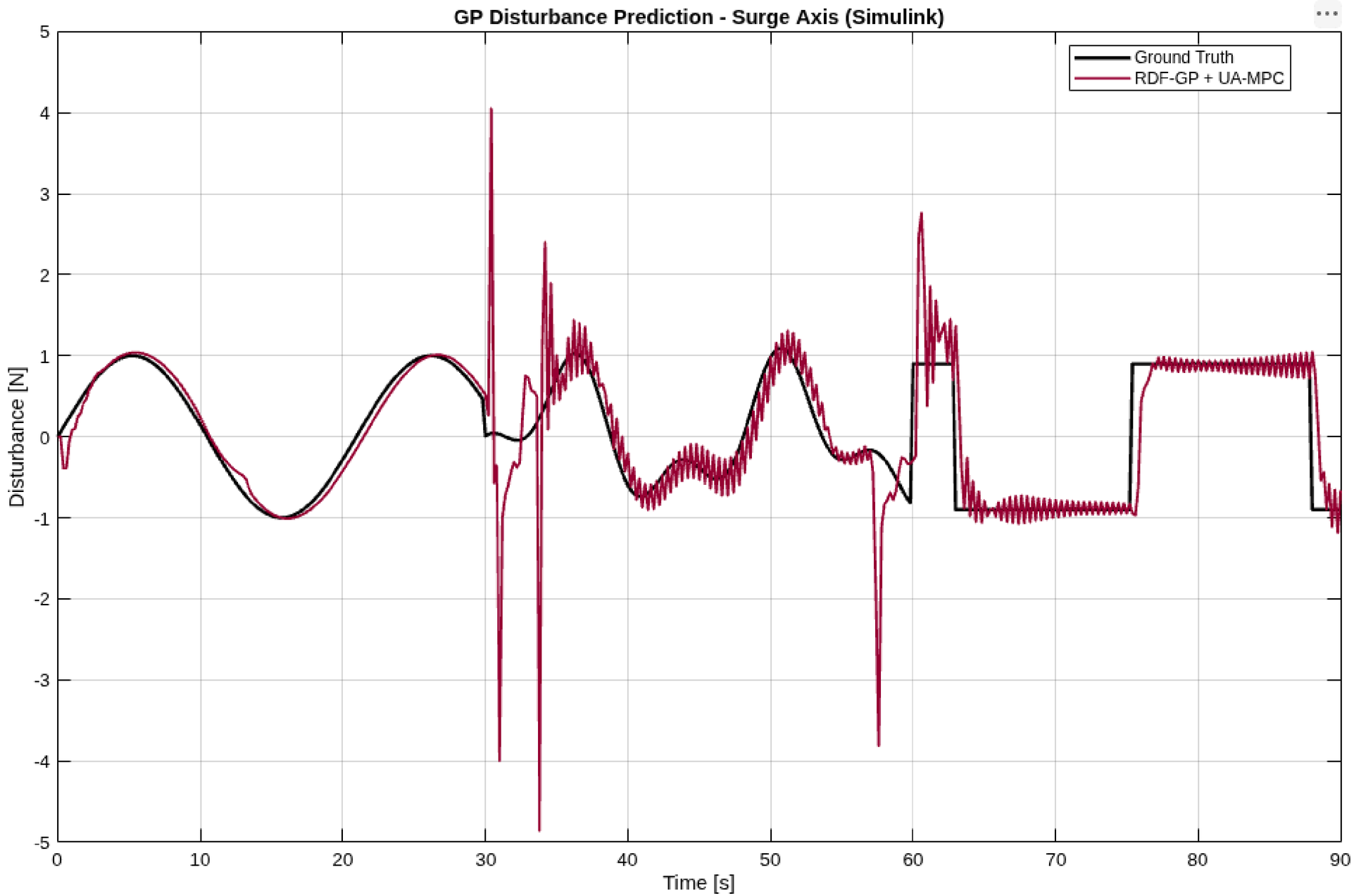
Tracking Error Before / After Fault

Shows how much the tracking error grows after the yaw thruster drops to 50% at t = 45 s, and which method copes better.



Simulink Disturbance Prediction, Surge Axis

Shows that the GP learns the disturbance the same way inside the Simulink model as in the MATLAB simulation.



Results 1: Five-Method Comparison

Method	Tracking RMSE (m)	Disturbance RMSE
NoGP	0.2126	N/A
StaticGP	0.1995	0.5597
DFGP_LP (base paper)	0.1999	0.6157
RDFGP_UAMPC (ours)	0.1999	0.5987
PID	0.2447	N/A

Inference from the above table:

- **PID is worst (0.2447 m); even NoGP beats it.**
- **StaticGP is slightly best in this one run; our edge shows up across many runs.**
- **PID and NoGP have no estimator, so no disturbance score.**

Results 2: PID vs Learning-Based MPC

Property	PID	Our MPC
Control	Reactive	Predictive
Model	None	4-DOF inside optimiser
Disturbance	Ignored	Learned online
Limits	Clamped afterwards	Built into optimisation
Axis coupling	Ignored	Modelled
Tracking RMSE	0.2447 m	0.1995–0.2126 m

Inference from the above table:

- **Each design difference shows up as better tracking.**
- **NoGP alone beats PID, so prediction helps even without learning.**
- **Caveat: PID gains were hand-tuned.**

Results 3: Base Paper vs This Project

Method	Base Paper (Prediction)	Proposed Work	Δ	Paper (Track)	Proposed Work	Δ
No GP	—	N/A	—	0.1050	0.2126	+102%
DF-GP (LP)	0.289	0.616	+113%	0.0285	0.1999	+601%

Inference from the above table:

- **Proposed work errors are higher because our controller's model equals the simulated vehicle, so there is little for the GP to correct.**
- **The paper had a real mismatch (simulator and pool tests).**
- **Proposed work did not tune the plant to match their numbers; the fair test is the five-way comparison.**

Results 4: Monte Carlo (8 seeds, σ = 0.01)

Method	Pos. mean	Pos. std	Dist. mean	Dist. std
NoGP	0.1970	0.0003	N/A	N/A
StaticGP	0.2066	0.0028	3.1959	0.6602
DFGP_LP	0.2050	0.0030	3.4889	0.5590
RDFGP_UAMPC	0.2055	0.0024	3.6408	0.3384

Inference from the above table:

- **Learning methods are within 0.0016 m of each other on average.**
- **Ours has the lowest spread (0.3384 vs 0.6602): most consistent, not most accurate.**
- **Consistency matters for planned missions.**

Results 5: Thruster Fault (yaw 50% at t = 45 s)

Method	Before (m)	After (m)	Degradation
NoGP	0.1950	0.2632	+35.0%
RDFGP_UAMPC	0.1958	0.2583	+31.9%

Inference from the above table:

- **Both get worse after the fault.**
- **Ours degrades 3.1 points less with no fault-handling code.**
- **The controller sees a fault as just another disturbance, and the GP learns it.**

Results 6: Fusion Behaviour, LP vs QP

Property	LP (paper)	QP (ours)
Cost	Linear	Quadratic (ρ = 0.05)
Optimum	Corner	Can be inside
Behaviour	Hard switch	True blend
Churn per step	0.049	0.045

Inference from the above table:

- **Churn = how much the weights change each step; lower is steadier.**
- **A linear cost cannot blend; this is maths, not tuning.**
- **The drop is small (~8%); the weights graph is the main evidence.**

Novelties and Contributions

Five things this project adds beyond reproducing the paper.

Fix and use	Test and compare	Make it trustworthy
1. Blend repaired: hard switch → true blend (churn 0.049 → 0.045)	3. First PID benchmark: 0.2447 m vs 0.1995–0.2126 m	5. Sign error fixed; two scripted Simulink models
2. Variance used: uncertainty tightens thrust limits	4. Fault + Monte Carlo: +31.9% vs +35.0%; std 0.3384 vs 0.6602	Every claim is measured, not asserted

Results 6: Fusion Behaviour, LP vs QP

Sectors: subsea energy · offshore wind · ports and defence · tethered ROVs

Problem	Our answer	Evidence
Survey quality drops in moving water	Learn the disturbance online	0.1995–0.1999 m
Disturbances act on several timescales	3 GPs blended every step	Churn 0.049 → 0.045
Vehicles use PID, benefit assumed	Measured under same conditions	PID 0.2447 m
Thrusters weaken mid-mission	Learning absorbs part of the loss	+31.9% vs +35.0%
Noisy sensors	Most consistent response	std 0.3384 vs 0.6602
Controllers must be auditable	Scripted Simulink models	Both run and agree

Base Paper vs Proposed Work

Same in both: 4-DOF model · RK4 · MPC cost and constraints

Aspect	Base paper	This project
Validation	Simulator + real BlueROV2	MATLAB + Simulink only
Solver	qpOASES	fmincon + quadprog
GP type	Sparse (inducing points)	Dense weighted
Hyperparameters	Iterative fitting	Direct formula
Data and input	Static + sliding; history window	Sliding only; current state
Weight fusion	LP: hard switch	QP: true blend
Variance	Computed, unused	Tightens thrust limits
Testing	Single run, no PID	PID, fault, 8-seed Monte Carlo

Overall Comparison with the Base Paper

Metric	Base paper	This project	Status
Disturbance RMSE (DF-GP)	0.289	0.616	Reproduced
Tracking RMSE (DF-GP)	0.0285	0.1999	Reproduced
Tracking RMSE (No GP)	0.1050	0.2126	Reproduced
Weight fusion	Hard switch	True blend	Corrected
Fused variance	Unused	Tightens limits	New
PID comparison	None	0.2447 vs 0.1995–0.2126 m	New
Thruster fault	None	+31.9% vs +35.0%	New
Robustness	Single run	8 seeds, std 0.3384 vs 0.6602	New

Limitations

Testing limits	Method limits	Baseline limit
1. Simulation only: no pool or hardware test	2. Overshoot loop at the sharpest curve; two fixes made it worse, so reverted	4. PID tuned by hand: better tuning could narrow the gap
Next step: pool test on a real vehicle	3. Not always most accurate: advantage is consistency, not lowest average error	Next step: tune PID by a standard method and re-compare

THANK YOU

- Presented by Team - A16